

Document Number: P2995R0

Date: 2023-10-07

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2023-05-24 through 2023-09-27

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [May 24th, 2023](#)
- [June 7th, 2023](#)
- [July 12th, 2023](#)
- [July 26th, 2023](#)
- [August 23rd, 2023](#)
- [September 13th, 2023](#)
- [September 27th, 2023](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)
- [P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26](#)
- [P2512: SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15](#)
- [P2605: SG16: Unicode meeting summaries 2022-01-12 through 2022-06-08](#)
- [P2678: SG16: Unicode meeting summaries 2022-06-22 through 2022-09-28](#)
- [P2766: SG16: Unicode meeting summaries 2022-10-12 through 2022-12-14](#)
- [P2891: SG16: Unicode meeting summaries 2023-01-11 through 2023-05-10](#)

May 24th, 2023

Draft agenda:

- [P2779R0: Make basic_string_view's range construction conditionally explicit](#).
- [P2863R0: Review Annex D for C++26](#).
- [P2871R0: Remove Deprecated Unicode Conversion Facets From C++26](#).
- [P2873R0: Remove Deprecated Locale Category Facets For Unicode from C++26](#).
- [P2872R0: Remove wstring_convert From C++26](#).

Attendees:

- Alisdair Meredith
- Charlie Barto
- Corentin Jabot
- Eddie Nolan
- Fraser Gordon
- Giuseppe D'Angelo
- Jens Maurer
- Mark de Wever
- Mark Zeren
- Peter Bindels
- Peter Brett
- Robin Leroy
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2779R0: Make basic_string_view's range construction conditionally explicit](#):
 - Giuseppe presented an overview of the paper including relevant history:
 - [P1989R2 \(Range constructor for std::string_view 2: Constrain Harder\)](#) added an implicit `std::string_view` constructor that enables implicit conversion from any type that satisfies a set of constraints, one of which includes having a member type alias named `traits_type` that matches the `std::string_view` member of the same name.
 - [P2499R0 \(string_view range constructor should be explicit\)](#) changed the new constructor to be declared `explicit` due to concerns involving ranges that do or do not contain an embedded null character; this broke the ability for string types to implicitly convert to `std::string_view`.
 - [LWG 3857](#) removed the constraint requiring a matching `traits_type` member type alias based on the rationale that such a safety precaution is no longer necessary since conversions are now explicit.

- The proposed paper seeks to conditionally restore implicit conversions for string-like types without requiring modifications to those types to add conversion operators.
- Two options are proposed:
 - Option 1 adds an opt-in trait and makes the constructor conditionally explicit based on the presence of a matching member `traits_type` type alias.
 - Option 2 makes the constructor conditionally explicit based on the presence of a matching member `traits_type` type alias without requiring an opt-in trait.
- Qt has provided a `QStringView` class with an [implicit constructor that accepts a range](#) that has worked well in practice for a decade.
- PBrett asked what the essential nature of a string-like type is.
- Giuseppe responded that it is a contiguous sequence of characters and associated character classification traits.
- PBrett argued for substitution of "code units" for "characters".
- Zach noted that the `traits_type` name might be used by types that are not string-like types, stated that he does not typically add a `traits_type` to his own string-like types, and asked what is commonly done in practice.
- Giuseppe responded that the paper lists the results of a survey of various projects for occurrences of the `traits_type` name and found that it is strongly correlated with string-like types but that there are string-like types that don't have such a member.
- Giuseppe acknowledged that the `traits_type` name is quite generic.
- Victor expressed opposition to option 2 since it relies on what he considers to be a legacy feature and that `traits_type` is, in practice, always `std::char_traits`.
- Victor asserted that implicit conversions and implicit interoperation with the standard library are not desired for Folly's `fbstring`.
- Victor stated that he is ok-ish with option 1.
- Tom asked Victor to further explain his concerns and the damage he fears the implicit conversions would cause.
- Victor replied that use of `fbstring` is no longer encouraged and the proposed change would facilitate continued usage.
- Victor noted that the proposed changes could also impact overload resolution in generic code and potentially introduce overload resolution failures due to ambiguity.
- Corentin lamented the ability for programmers to specialize `std::char_traits` for their own user-defined types and stated he plans to propose deprecating or removing that allowance.
- Corentin explained that the interface that `std::char_traits` provides is not a good match for how text processing works in practice.
- Corentin asserted that increased use of `std::char_traits` should be discouraged.
- Corentin opined that option 1 is fine but that option 2 is problematic in the long run.
- Giuseppe acknowledged Corentin's position.

- Corentin clarified that programmers should not be encouraged to use a different type than `std::char_traits` but rather that they should be encouraged not to use a `char_traits`-like type at all.
- Tom summarized his understanding of the concerns; the proposed change could encourage programmers to add a `traits_type` member type alias of `std::char_traits` to classes that otherwise wouldn't define the type alias solely to enable implicit conversions to `std::string_view`.
- Zach argued for not enabling such implicit conversions at all on the basis that `std::string_view` is intended to be implicitly convertible from other standard library types and that explicit conversions are appropriate elsewhere.
- Alisdair opined that the right approach would be for types to opt themselves in to an implicit conversion.
- Alisdair asserted that `std::char_traits` is not legacy and that it cannot be removed without significant ABI impact.
- Alisdair stated that the matching `traits_type` constraint is a good heuristic and that the opt-in trait in option 1 is so specific that he would have a hard time supporting it.
- Jens noted that the proposed wording for option 1 requires both the opt-in string-like-type trait and the matching `traits_type` constraint to enable implicit conversions.
- Jens expressed a preference for an option that proposed only the string-like-type trait.
- Jens stated that the wording needs to be rebased on the current working paper since the struck wording has already been removed.
- Jens suggested `is_string_view_like` might not be the best choice of name for the opt-in trait and suggested `enable_view` as an example name for similar opt-in traits.
- Giuseppe acknowledged the suggestion and stated that the name can be changed.
- Jens noted that it doesn't matter how string-view-like the source type is as long as it provides contiguous storage and opts itself in.
- Jens agreed with not wanting to encourage the addition of an otherwise unused `traits_type` member.
- Jens observed that `is_string_view_like` is false by default.
- Jens suggested that, if it is desirable to provide a safety check on a matching `traits_type` member, that the `is_string_view_like` trait can support a mechanism to enable that.
- Jens expressed a preference for postponing a poll to forward the paper until it has been rebased on the current working paper.
- Various poll options were discussed but it was decided that polling be postponed pending an updated paper revision with wording rebased on the current working paper and an additional option to enable implicit conversions based solely on the opt-in trait.
- [P2863R0: Review Annex D for C++26](#):
 - Alisdair introduced this and the following papers.
 - Tom explained his understanding of the ramifications for removal of standard library features; that an implementor may choose not to provide the removed features or

may choose to provide them since the removed names are reserved as "zombie" names.

- Alisdair acknowledged the intent, but noted that the standard currently lacks wording to support zombification of explicit template specializations.
- Alisdair explained that there are four deprecated subclauses that are relevant to SG16; [D.26 \(\[depr.locale.stdcvt\]\)](#), [D.27 \(\[depr.conversions\]\)](#), [D.28 \(\[depr.locale.category\]\)](#), and [D.29 \(\[depr.fs.path.factory\]\)](#).
- PBindels stated that [D.15 \(\[depr.str.strstreams\]\)](#) and [D.25 \(\[depr.string.capacity\]\)](#) have to do with text facilities but that he reviewed them and concluded that the functionality is not strongly relevant for SG16.
- Alisdair stated that, for `std::filesystem::u8path`, per [LWG 3840](#), there have been recent comments that removal would be problematic.
- Tom stated that the LWG issue was recently discussed in LEWG but that the LWG issue does not appear to have been updated to reflect that discussion.
- *[Editor's note: LEWG discussed the LWG issue during its [2023-01-10 telecon](#).]*
- Alisdair stated that deprecated features should either be undeprecated or removed and noted that this feature has been deprecated since C++20.
- Jens expressed concern regarding Billy O'Neal's comment in the LWG issue that deprecation of `u8path` was one of the reasons that `vcpkg` discontinued use of `std::filesystem`.
- Jens stated that SG16 should offer an opinion.
- Corentin replied that there was a poll in LEWG in January and that there was no consensus to undeprecate `u8path`.
- Corentin stated that a mechanism to access a sequence of `char` that holds UTF-8 code units as-if it were a sequence of `char8_t` is a feature that we should have; we're missing a way to pass such a sequence to the `std::filesystem::path()` constructor such that it is interpreted as UTF-8.
- Tom noted that Corentin has a paper on that topic.
- *[Editor's note: See [P2626 \(charN_t incremental adoption: Casting pointers of UTF character types\)](#).]*
- Alisdair noted that, if removed, `u8path` would be added to the list of zombie names, so implementors that wish to continue providing it may do so.
- PBindels opined that `u8path` provides a solution to work around legacy issues but that Corentin's P2626 provides a proper solution.
- PBindels suggested that we should neither undeprecate nor remove `u8path` until a proper solution is in place.
- Alisdair stated that he can update the paper to reflect that guidance and to note further action as dependent on P2626.
- Charlie agreed with not removing `u8path` without a proper alternative.
- Charlie noted that, if `u8path` is zombified, that implementors can continue to provide it, but that portability is lost.
- Charlie stated that he didn't see a reason to remove `u8path`; that it isn't harmful.
- Alisdair acknowledged that a migration path is needed.

- Tom explained that the original motivation for deprecation was to dissuade continuing to provide standard library functions that require UTF-8 data in char-based storage.
- Tom noted that `u8path` and the deprecated `std::codecvt` facets were the only standard library features that did so.
- [P2871R0: Remove Deprecated Unicode Conversion Facets From C++26:](#)
 - Alisdair presented the paper:
 - These facets were deprecated because they did not provide error handling capabilities and could not reasonably be extended.
 - There are some implementations that do not issue deprecation warnings.
 - Corentin noted the work in progress and general plan to provide replacements for C++26 and suggested waiting to remove them pending that work.
 - Jens agreed and stated that removal without replacements is ill-advised unless these are actively causing harm.
 - Tom noted that conversions are possible through the `mbrtoc*` and `c*rtomb` family of functions though those have their own issues.
 - Victor stated that the `codecvt` facets are so challenging to use that not having a replacement isn't really a problem.
 - Alisdair noted that implementors can continue to provide them thanks to zombification.
 - Alisdair reported that, per the paper, LEWG and SG16 previously recommended removal during the C++23 cycle, but that action wasn't completed.
 - Alisdair reminded the group that `codecvt_utf` and `codecvt_utf1` convert to and from UCS-2 or UTF-32 depending on the size of the first template parameter.
 - PBrett asked for any objections to removal.
 - No objections were reported.
 - Alisdair stated he will take that feedback back to LEWG.
- [P2873R0: Remove Deprecated Locale Category Facets For Unicode from C++26:](#)
 - Tom explained that these facets were deprecated because they convert to and from UTF-8 in char-based storage rather than between the multibyte encoding like the non-deprecated facets do.
 - Tom reported that `char8_t`-based replacements were added as replacements, but those were a mistake because they won't be used by char-based streams anyway.
 - [*Editor's note: [LWG 3767](#) tracks deprecating the `char8_t`-based facets.*]
 - PBrett asked for any objections to removal.
 - No objections were reported.
 - Corentin spoke in favor of removal.
- [P2872R0: Remove `wstring\_convert` From C++26:](#)
 - Giuseppe asked if the paper includes removal of `std::wbuffer_convert`.
 - Alisdair confirmed that it does.
 - Alisdair explained that these were deprecated because the example for `std::wstring_convert` used another deprecated feature, `std::codecvt_utf8` and, due to other underspecification concerns, no one was motivated to fix them.

- Alisdair asked if SG16 is the right group to address this.
- PBrett responded affirmatively and stated that SG16 is the group that misunderstands `wchar_t` the least.
- Alisdair noticed some issues with the paper and concluded that updates are required before the paper is ready for any action to be taken on it.
- Tom stated that the next meeting is tentatively scheduled for 2023-06-07 and will likely continue review of [P2779 \(Make basic_string_view's range construction conditionally explicit\)](#) and [P2872 \(Remove wstring_convert From C++26\)](#) if updated revisions are available followed by an initial review of [P2845 \(Formatting of std::filesystem::path\)](#).
- Zach reported that he expects to have a new revision of [P2728 \(Unicode in the Library, Part 1: UTF Transcoding\)](#) available soon after the Varna meeting.

June 7th, 2023

Draft agenda:

- [P2779R1: Make basic_string_view's range construction conditionally explicit.](#)
- [P2872R1: Remove wstring_convert From C++26.](#)
- [P2845R0: Formatting of std::filesystem::path.](#)

Attendees:

- Alisdair Meredith
- Charlie Barto
- Corentin Jabot
- Fraser Gordon
- Giuseppe D'Angelo
- Jens Maurer
- Mark de Wever
- Mark Zeren
- Peter Brett
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2779R1: Make basic_string_view's range construction conditionally explicit.](#)
 - *[Editor's note: D2779R1 was the active paper under discussion at the telecon. The agenda and links used here reference P2749R1 since the links to the draft paper*

were ephemeral. The published document may differ from the reviewed draft revision.]

- Giuseppe summarized the paper and changes since the last revision:
 - The paper endeavors to identify a compromise position for the issues that have resulted in multiple changes to how the `std::basic_string_view` range constructor is specified.
 - Option 2 from the previous revision is still present though there was not much support for this option in the last discussion.
 - Option 1 follows existing precedent for type traits that enable some functionality; this option has been divided into two sub-options.
 - Option 1-A provides a type trait that enables conversion without regard to the `traits_type` member.
 - Option 1-B provides the type trait from option 1-A as well as an additional type trait that can be used to enable conversion that is sensitive to the `traits_type` member.
- Tom asked if the intent is for the trait to be used only for conversion to `std::string_view` or for conversion to any `string_view`-like type.
- Giuseppe responded that it is intended to be used for conversion to any `string_view`-like type.
- Jens suggested in chat: "You can also define `enable_string_view_conversion` in a way so that the user specialization can compare `char_traits`, if so desired (or not)."
- Jens' suggestion received several positive responses.
- Alisdair, following up on Jens' suggestion in chat, asked if the traits in option 1-B could be merged.
- Giuseppe confirmed that they could be.
- Alisdair indicated that would be his preference.
- Alisdair stated that the conversion could be enabled based on a class member similar to how transparent key comparison for associative containers is enabled via the `is_transparent` member of the `compare` class.
- Giuseppe acknowledged that approach would work as well.
- Tom noted that approach would require modifying the class.
- Alisdair responded that the trait could still be specialized but could be defaulted based on the presence of a member.
- Jens stated that the most convenient option would be to define a conversion operator with the trait available as a fallback.
- Jens expressed a preference for a single trait with template parameters such that a specialization can be written to explicitly match `traits_type` or `std::char_traits` as desired.
- Jens noted that `enable_string_view_conversion_with_traits` still requires comparison with `std::char_traits` or a `traits_type` member.
- Jens suggested that third party `string_view`-like classes can provide their own trait to enable implicit conversions.

- Giuseppe responded that the goal is to enable interconvertibility between different string types.
- Giuseppe noted that the proposal doesn't require comparisons with specific type or member names.
- Zach stated that he doesn't find the problem that the paper intends to address compelling and noted that `std::string_view` is available as a vocabulary type.
- Zach noted that working around the lack of an implicit conversion just requires slightly more code; explicit construction of a `std::string_view` object.
- Victor requested that the two traits in option 1-B be merged.
- Victor agreed with Alisdair's suggestion to default the trait to enable based on the presence of a class member.
- Victor asserted that only the author of a class should opt a class into the proposed behavior; not users of the class.
- Victor repeated his opposition to enabling implicit third party interoperation.
- Corentin stated that most of the proposed behavior should be being discussed in LEWG rather than in SG16 and that SG16 just needs to provide a recommendation whether use of `std::char_traits` is a good heuristic.
- PBrett responded that there is an SG16 question concerning which types are sufficiently text-like.
- PBrett asked for poll suggestions.
- Tom noted that discussion revealed other options that should be explored.
- Tom suggested polling the desire to enable interconvertibility across any/all string-like types in the ecosystem.
- Poll wordsmithing ensued.
- **Poll 1.1: Any opt-in to implicit range construction of `std::string_view` should be explicit on a per-type basis.**
 - **Attendees: 12 (1 abstention)**
 - **SF F N A SA**
2 8 0 1 0
 - **Strong consensus.**
 - **A: If types have character traits, we should be making use of them to determine compatibility.**
- Jens responded to the against rationale by stating that use of character traits is not excluded; per-type enablement could be conditional on matching traits.
- **Poll 1.2: The standard library should provide a general-purpose facility for enablement of implicit interconvertibility between string and string_view-like types (including UDTs).**
 - **Attendance: 12 (2 abstentions)**
 - **SF F N A SA**
1 1 4 4 0
 - **No consensus.**
- **Poll 1.3: A solution to the problem stated in P2779 needs to be included in the C++ standard library.**

- **Attendance: 12 (1 abstention)**

- **S F F N A S A**

- 1 1 5 4 0**

- **No consensus.**

- Tom stated that he will record the poll results in the paper tracker and that it will be up to the LEWG chair to decide what to do next.
- PBrett suggested that more examples of how this proposal could alleviate programming challenges
- might help to increase motivation.
- Tom agreed and noted that the large proportion of N votes presumably reflects insufficient motivation.

- [P2872R1: Remove `wstring\_convert` From C++26.](#)

- *[Editor's note: D2872R1 was the active paper under discussion at the telecon. The agenda and links used here reference P2872R1 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
- Alisdair stated that, If feedback is light, that he will incorporate it and publish the paper as P2872R1; otherwise, he will publish P2872R1 as-is and incorporate the feedback in a newer revision.
- Alisdair explained that `wbuffer_convert` and `wstring_convert` have been deprecated for three standard releases now.
- Alisdair noted that removal permits implementors to continue to provide the functionality thanks to the additions to zombie names.
- Alisdair indicated that wording updates might be needed, but that LWG will handle that.
- Alisdair explained that the deprecation was motivated by underspecification and dependence on other deprecated features like `std::codecvt_utf8`.
- Alisdair reported that there are currently four related open LWG issues and that reviving the feature would require more.
- Corentin stated that, without `std::codecvt_utf8`, the standard no longer provides features needed to use these types.
- Alisdair agreed and explained that programmers would have to provide their own `std::codecvt` facet.
- Corentin acknowledged the requirement, but observed that programmers could more easily just implement the needed conversion.
- Victor opined that these types provide little value since they are just light wrappers anyway.
- Victor reported that a search of the projects he works on found a few uses, but that those uses should be replaced anyway.
- PBrett asked if anyone had an objection to removing these features.
- No objections were raised.
- MarkZ reported that a Github search identified few uses.

- [P2845R0: Formatting of `std::filesystem::path`.](#)

- Victor introduced the paper:
 - [P1636 \(Formatters for library types\)](#) previously proposed formatting for `std::filesystem::path` but was specified to use the `native()` member function which might require transcoding and had no provisions for handling of non-printable characters.
 - This paper proposes a formatter that performs proper transcoding and substitutes escape sequences for non-printable characters and ill-formed code units.
- Victor noticed a missing doublequote character in the first source code example in section 2, "Problems".
- Victor reported that some minor issues have been fixed in a draft R1 revision.
- Corentin asked if backslash path delimiters on Windows would be formatted with escape sequences.
- Victor confirmed that they would be, that such substitution might be surprising, but is consistent with `std::quoted()`.
- Victor noted that an additional format specifier could be provided to choose an alternate behavior.
- Corentin asked about use of the debug specifier, "{:?}",
- Victor replied that the escaped format is proposed as the default behavior.
- Charlie asserted that some latitude is needed to choose an alternate escape character since backslash in paths has an important meaning on Windows.
- Charlie noted that an alternate escape character could be surprising and would create an inconsistency across platforms.
- PBrett asked about adding a specifier that enables specifying a different escape character.
- Victor responded that such a specifier would be cumbersome and that there are other options such as performing a transformation.
- Victor stated that there are use cases for both an escaped and a non-escaped variant.
- Tom presented a few use cases including formatting for generic text, byte preserved for filesystem access, punycode for URLs, and quoted for shell scripts.
- Tom suggested that most transformations should be done outside of formatting.
- Corentin stated that the default behavior should just escape ill-formed code units and that the debug format specifier could be used to escape problematic characters.
- Victor replied that quoting is useful but not always needed.
- Tom suggested that a specifier could be added to opt in to quoting.
- PBrett expressed two high level use cases:
 - The need to format the path precisely such that it can be used to open a file.
 - The need to format the path for textual display in a format friendly to humans.
- PBrett opined that the paper does not clearly define the problem it intends to solve.
- PBrett noted that, in [GLib](#), functions are provided to request a file name suitable for display as valid UTF-8 or as a byte array.

- Victor replied that the goal of the paper is to address the issues discovered from prior review of [P1636 \(Formatters for library types\)](#).
- Victor stated that additional use cases can be addressed as needed.
- Zach reported that Python provides the functionality this paper is proposing and noted that its formatters will double Windows path separators.
- Zach stated that Python allows printing unformatted paths by treating paths as a string and that C++ can do so as well.
- Zach agreed that some kind of escaping and quoting is needed.
- *[Editor's note: Corentin later [posted a message to the SG16 mailing list](#) that demonstrates Python's behavior with a [Compiler Explorer link](#).]*
- Jens asserted that, due to various quirks with `std::filesystem::path`, that the paper should cover the motivation and design space and not solely focus on addressing the issues found from review of P1636.
- Jens stated that the paper should discuss, for example, the implication of using backslashes in the syntax of character escapes in formatted paths.
- PBrett agreed.
- PBrett noted that we were out of time and that additional review will be needed to discuss encoding issues.
- Tom stated that the next meeting is scheduled for 2023-06-28, that there are several LWG issues awaiting review, and that Zach is working on a revision of [P2728 \(Unicode in the Library, Part 1: UTF Transcoding\)](#).
- *[Editor's note: The following meeting was canceled due to summer vacations.]*
- Zach stated an expectation to have a new revision available in the next two weeks.

July 12th, 2023

Draft agenda:

- [P1030R5: std::filesystem::path_view](#):
 - Discuss what to do in lieu of overloads with `std::locale` parameters.
- [P2845R0: Formatting of std::filesystem::path](#):
 - Continue review.
- [LWG 3944: Formatters converting sequences of char to sequences of wchar_t](#):
 - Initial review.

Attendees:

- Charlie Barto
- Fraser Gordon
- Hubert Tong
- Jens Maurer

- Mark de Wever
- Nathan Owen
- Niall Douglas
- Peter Brett
- Robin Leroy
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P1030R5: std::filesystem::path_view](#):
 - Niall stated that, during LEWG discussion in Varna, LEWG approved removal of `std::locale` function overloads that were added for compatibility with `std::filesystem::path`.
 - Niall noted that, for each overload set that has an overload with a `std::locale` parameter, there is an overload that does not.
 - PBrett asked for an explanation of the concerns with the overloads that work with `std::locale`.
 - Niall responded that locale support generally delegates conversion to the OS where they are handled efficiently, but conversions performed via `std::locale` impose considerable performance overhead; possibly including multiple conversions on some platforms.
 - *[Editor's note: conversions controlled by `std::locale` require use of the `std::codecvt` facet which, per [\[fs.path.construct\]p6](#), may require multiple conversions.]*
 - Niall stated that a replacement for `std::locale` would be welcome.
 - PBrett opined that, in his experience, treating paths as having an encoding leads to sadness.
 - PBrett stated that a lossy conversion to a definitive encoding can be used to display paths.
 - Niall noted that the proposed `path_view` supports a raw byte encoding and provides rendering operations.
 - PBrett asked if the facility provides features to produce a path suitable for display purposes.
 - Niall replied that such formatting falls more in the domain of [P2845 \(Formatting of std::filesystem::path\)](#) and that he has been in discussion with Victor.
 - PBrett asked if there is a plan to provide a formatter for `path_view`.
 - Niall suggested that such a formatter behave the same as for `std::filesystem::path`.
 - Victor summarized observations made during the LEWG discussion:
 - `std::locale` was present in `constexpr` overloads; that issue is easily solved by removing the `constexpr` specifier from those declarations.

- the `std::locale` parameter is only present to support encoding conversions, but those conversions are better handled by an interface designed for such conversions.
- Victor noted that `std::codecvt` is not an efficient method for transcoding.
- Victor opined that the overloads with a `std::locale` parameter are not known to be needed and can be added back later, perhaps in a more restrictive form, if desired.
- Niall asked Victor if he is suggesting that the existing `std::filesystem::path` overloads with a `std::locale` parameter should be deprecated.
- Victor replied that he would be happy to write such a paper at some future point.
- Tom asked why there is a `compare()` overload with a `std::locale` parameter.
- Niall responded that comparisons are shallow by default and `compare()` is provided to allow for more comprehensive equivalence comparisons.
- Niall explained that the `std::locale` parameter is used to convert each path to a common form that is then compared.
- PBrett expressed an assumption that the `std::locale` parameter would be used for collation purposes using the `std::collate` facet.
- Hubert asked why collation would be relevant for equality.
- PBrett asked if, given a set of `path_view` objects, whether the `compare()` operation could be used to order them.
- Zach responded that such collation might be better performed using features outside of the `std::filesystem` library.
- Jens stated that the wording in the paper is suggestive that only the encoding is intended to be consumed from the locale object.
- Jens observed that removal of the `std::locale` parameter results in a loss of transcoding facilities, but since what was provided was so thin, it isn't much of a loss.
- Victor stated that the equivalent facility in `path_view` of the `std::locale` based `std::filesystem::path` construction is the locale dependent `render()` member function.
- Niall explained that the reference implementation of the locale dependent `render()` member uses the `std::locale` object to convert a path to UTF-8 and then compares it.
- Tom expressed confusion, stated that `std::locale` doesn't support conversion to UTF-8, and then realized the reference implementation is probably using the `char8_t` `codecvt` facets that don't actually convert between the locale encoding.
- Niall responded that he is not aware of anyone that uses `std::locale` with the `filesystem`.
- Victor pondered interaction with `std::format` and `std::print` and whether it would make sense for `path_view` to also rely on the literal encoding to detect UTF-8 encoding; that would enable construction with `char`-based data to be saved as `char8_t`.
- Tom expressed some reservations; programmers might compile with a `/utf-8` or equivalent option, but file names produced or provided at run-time might be

differently encoded.

- Hubert expressed concerns regarding implementation experience obtained so far regarding preservation of the literal encoding for use by the standard library.
- **Poll 1: Modify P1030R6 "std::filesystem::path_view" to restore function overloads with locale parameters.**

- **Attendees: 12 (4 abstentions)**

- **SF F N A SA**

- **0 0 2 4 2**

- **Consensus against.**

- [P2845R0: Formatting of std::filesystem::path:](#)

- Tom apologized for his delinquency in producing a meeting summary for the previous discussion on this paper that took place at the prior SG16 meeting.
- Victor summarized his understanding of the direction from the prior meeting; to explore more options for quoting and escaping.
- PBrett explained a desired ability to obtain a close approximation of a path validly encoded for display purposes and stated that the paper does not currently provide sufficient detail.
- Victor asked for confirmation that Peter wants the path formatted without any transformation, no loss of information, no quoting, and perhaps just escaping for invalid code unit sequences.
- PBrett explained that he wants three version:
 - one that provides the raw bytes; `path_view` provides that, but `std::filesystem::path` does not.
 - one that understands encoding and provides the path unmodified with the exception of substitution characters for invalid code unit sequences.
 - one with quotes and escape sequences for problematic characters.
- Niall stated that, for both `std::filesystem::path` and `path_view`, it is possible to obtain the path as a string or to visit the components with a lambda.
- Jens asked for confirmation that `std::format` includes a debug specifier that enables a string to be printed with escape sequences for problematic characters.
- Victor confirmed that is the case and stated that it could be used for paths such that the default formatting provides the second option PBrett listed.
- Jens asked what the output would be for the Belarusian example in the paper for arbitrary code pages used in practice.
- Victor replied that, in either case, the same substitutions would be performed.
- Jens expressed approval and noted that behavior would be consistent with choices previously made.
- Mark observed that the options discussed so far, with an exception for the debug specifier, would retain newline characters.
- PBrett acknowledged the behavior and noted that additional translations can be applied on the formatted result as needed; e.g., to substitute a space for the newline character.

- Niall expressed frustration regarding rendering paths in quotes since quote characters are also valid path characters.
- Tom acknowledged feeling similarly frustrated by that.
- PBrett stated that quotes would only be present when the debug specifier is used.
- Niall pondered whether an additional format specifier to format the path with escape sequences but without quotes is warranted.
- Tom responded that additional such options could be recognized by the `formatter` specialization.
- Zach asked how control characters like RTL isolates should be handled; whether they should be ignored when formatting for display but preserved by the debug format.
- PBrett replied that he doesn't have experience with those in path names but that he would expect them to be handled as a custom translation.
- Zach suggested such characters should probably be passed through when formatting for display.
- PBrett asked if the paper should be updated to address the `path_view` proposal.
- Victor replied that `path_view` should be handled separately since there are additional complications for the byte case.
- Tom stated that the consensus direction seems pretty clear for a paper revision.
- [LWG 3944: Formatters converting sequences of char to sequences of wchar_t:](#)
 - Mark summarized the issue:
 - In C++20, it was an intentional design decision to not support formatting of char-based string arguments when formatting for `wchar_t`.
 - In C++23, such formatting was inadvertently added via support for range formatting since a range might have a `char` element type.
 - PBrett asked Mark what his preferred resolution is.
 - Mark replied with a preference to preserve formatting of individual characters of type `char` in general but to disable formatting of ranges with a `char` element type.
 - Mark noted that such range formatting probably wouldn't produce the intended result when the characters are, for example, individual UTF-8 code units.
 - PBrett expressed skepticism that the reported formatting was intentional.
 - Tom asked why a different conclusion is reached for formatting of an individual character vs an individual character in a range.
 - Hubert replied that a range of individual code units is more string-like.
 - Niall stated that, in principle, the range could be iterated to decode characters.
 - PBrett agreed but noted that doing so would require encoding information.
 - Niall acknowledged the requirement and noted it could be inferred for the `charN_t` types, but not for `char`.
 - Tom expressed a belief that support for the `charN_t` types is disabled.
 - Victor confirmed that is the case.
 - Hubert indicated that such conversions could be enabled, but that necessary facilities are not currently available at run-time; something like ICU or `iconv` would be needed.

- PBrett suggested that an escape translation could be produced.
- Hubert replied that stateful encodings would require representing state.
- Tom asked what the downside is of disabling support for ranges that have a mismatched character type as the element type.
- PBrett replied that, ideally, it should be possible to format everything.
- Victor agreed with PBrett and stated that formatters for string-like types that have a mismatched character element type could be disabled and that a specifier to format a range as a string could be provided.
- Hubert expressed support for a protocol to opt-in to support of string-like types.
- Zach asked if `std::vector` would be considered a string-like type.
- Zach expressed support for disabling formatting of ranges with a mismatched character element type.
- Victor observed that disabling formatters for mismatched `std::string` and `std::string_view` would suffice to automatically disable types that derive from them.
- Victor expressed support for distinguishing between string-like and non-string-like types.
- Mark noted that support can always be added later for a disabled formatter and that disabling these formatters would be an improvement over the status quo.
- PBrett agreed and asked Mark if he is willing to author a proposed resolution.
- Mark agreed to do so.
- *[Editor's note: Mark offered a proposed resolution that is now reflected in the LWG issue.]*
- Tom announced that the next meeting will be 2023-07-26 and that the agenda will cover allowances for \$ in identifiers, encoding for the proposed `std::contracts::contract_violation::comment()` member function, and continued review of of Zach's UTF transcoding paper if a new revision becomes available.

July 26th, 2023

Draft agenda:

- [WG14 N3145: \\$ in Identifiers v2](#):
 - Determine whether a corresponding proposal for WG21 is desired.
- [P2811R7: Contract-Violation Handlers](#):
 - Discuss character encoding considerations for the `std::contracts::contract_violation::comment()` member function.
- [LWG 3944: Formatters converting sequences of char to sequences of wchar_t](#):
 - Continue review pending a proposed resolution or related paper.

Attendees:

- Corentin Jabot
- Eddie Nolan
- Hubert Tong
- Jens Maurer
- Joshua Berne
- Mark de Wever
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Ville Voutilainen
- Zach Laine

Meeting summary:

- [WG14 N3145: \\$ in Identifiers v2](#):
 - Hubert introduced the topic.
 - C23 explicitly blessed \$ as an allowed character in identifiers as an implementation-defined extension.
 - C has traditionally allowed this extension and support for it is widely implemented.
 - [P2342 \(For a Few Punctuators More\)](#) contains additional analysis.
 - Up to and including C++20, this has been a conforming extension in C++ since \$ in an identifier would be ill-formed.
 - In C++20, \$ is a UCN and combines with adjacent identifier characters to produce an ill-formed identifier.
 - In C++23, \$ is no longer a UCN and adjacency with identifier characters now yields two pp-tokens, the second of which renders the program ill-formed.
 - In C++26, \$ is a member of the basic character set, adjacency with identifier characters continues to yield two pp-tokens, but the \$ token may be discarded such that it is never processed during translation phase 7.
 - PBrett asked for clarification of what constitutes a conforming extension.
 - Corentin observed that this extension requires the production of a single pp-token when \$ is adjacent to an identifier character.
 - Corentin stated that sanctioning this allowance in the standard would restrict evolution of the language since it would prevent use of \$ as an operator.
 - Steve noted that the status quo is that all implementations allow \$ in identifiers by default, \$ is widely used in identifiers, and \$ appears in mangled names.
 - Steve stated that compilers are free to issue a diagnostic and produce a working executable for source code that is ill-formed according to the standard.
 - Hubert replied that the concern is with preprocessing; if \$ is not explicitly allowed in an identifier by the preprocessor, then it is handled as a separate token and the difference is observable.

- Hubert stated that issuing a diagnostic only during translation phase 7 would be difficult.
- Hubert asserted that wording changes are in order to continue to permit existing practice with \$ in identifiers.
- Hubert acknowledged concerns regarding how to word an allowance so that new uses of \$ are not restricted.
- Hubert noted that new uses are only problematic if they are not surrounded by whitespace.
- Jens suggested the possibility of reverting the adoption of [P258R2 \(Add @, \\$, and ` to the basic character set\)](#) for C++26.
- PBrett expressed opposition to doing so since that would contradict the direction established in WG14 and codified in C23.
- PBrett stated that this discussion is a good start regarding how to move forward.
- Jens opined that the WG14 rationale is not motivating and that he is therefore not motivated to follow the same direction in C++.
- Tom noted that there are backward compatibility concerns for some platforms due to use of \$ in identifiers in system headers.
- Corentin stated that the WG14 direction was to explicitly state that it is implementation-defined whether \$ is allowed in an identifier.
- Poll 1: Whether DOLLAR SIGN is accepted as an identifier start and/or identifier continuation character should be explicitly implementation-defined.

- **Attendees: 12 (4 abstentions)**

- **SF F N A SA**

- **1 2 1 2 2**

- **No consensus.**

- **SA: I don't think an identifier should be implementation-defined.**

- PBrett stated that the next step would be a proposal to EWG acknowledging the guidance here.
- Tom asked for opinions regarding the default modes of current compilers being non-conforming.
- Zach replied that all implementations offer an option to disable the extension.
- PBrett stated that every implementation is non-conforming in their default modes in practice.
- Corentin asserted that implementations should issue warnings for use of the extension.

- [P2811R7: Contract-Violation Handlers](#):

- Joshua introduced the topic:
 - SG21 is working on a specification for a contract violation handler.
 - The proposed `comment()` member function of `std::contracts::contract_violation` is intended to return a string containing the source code of the violated contract predicate.
 - The proposed encoding for the returned string is the ordinary literal encoding.
- Tom expressed support for use of the ordinary literal encoding.

- Tom asked if anything should be specified regarding handling of characters that are not encodeable in the ordinary literal encoding.
- Corentin agreed with use of the ordinary literal encoding on the basis that the text will be used at run-time.
- Steve asked for confirmation that the feature effectively converts a source code snippet to text.
- Joshua confirmed.
- Steve suggested that a hand wavy approach similar to that taken for `static_assert` is likely necessary except that the string has to survive until run-time and we lack a mechanism to communicate the encoding.
- Steve stated that the compiler should perform a best effort rendering in the target encoding with the understanding that, for example, an identifier might not be representable in Latin1.
- Jens observed that is a different operation than stringizing.
- Steve agreed.
- Corentin asked what the anticipated use cases are for the `comment()` function.
- Joshua replied that the primary use case is for logging; other use cases might involve using the result as a key for a map.
- Joshua asserted that it is not intended to provide source code that a programmer might expect to parse.
- Joshua stated that the output is only intended to be sufficient for a human to be able to correlate it with the original source code.
- Zach ruminated on the interaction of source encoding and literal encoding and how preprocessor stringifying works.
- Jens noted that the `assert` macro is similarly expected to embed source code in the output it produces.
- Jens stated that the wording for `assert` does not capture the fact that producing the output involves multiple transcoding steps.
- *[Editor's note: the transcoding steps are the conversion from the encoding of the input file ([\[lex.phases\]p1](#)) to the translation character set ([\[lex.charset\]p1](#)) then to the ordinary literal encoding ([\[lex.charset\]p8](#)) and then finally, if necessary, to the implementation-defined encoding used to write text to the standard error stream ([\[cassert.syn\]](#) via reference to the C standard).]*
- Jens observed that, for `comment()`, there is a possibility to differentiate these steps; the compiler performs the conversion to the ordinary literal encoding and the violation handler can then perform additional transcoding as necessary.
- Jens asserted that these are not novel problems.
- Jens observed that non-encodeable characters in string literals are ill-formed and that a preprocessor stringize operation that produces such a string would likewise be ill-formed.
- Jens posited doing similarly for contracts.
- Corentin stated that doing so makes sense and then described some additional encoding options:

- UTF-8 in `char8_t`, though that doesn't improve usability.
 - implementation-defined.
 - ordinary literal encoding with an escaping mechanism for non-encodeable characters.
- Corentin suggested it is likely best to just let implementors do what they think is best.
- PBrett stated that SG21 had strong consensus for the text returned by `comment()` being implementation-defined.
- PBrett noted that, since it is implementation-defined, there is no need to specify whether the content includes macro expanded text.
- PBrett asserted that it is essential that the encoding be specified and expressed support for the current paper direction.
- PBrett agreed that UTF-8 in `char8_t` is an option, but that the standard provides few facilities to consume it.
- Hubert noted that, since C does not prohibit non-encodeable characters in string literals, the stringize operation suffices for `assert` in C.
- Steve stated that it would be very suprising if a `char`-based string with an encoding other than the ordinary literal encoding was returned; a `char8_t`-based string should be used if a UTF-8 encoded string is always returned.
- **Poll 2: The value of `std::contract_violation::comment` should be a null-terminated multi-byte string (NTMBS) in the string literal encoding.**
 - **Attendees: 12 (1 abstention)**
 - **S F F N A S A**
8 3 0 0 0
 - **Unanimous consensus.**
- [LWG 3944: Formatters converting sequences of `char` to sequences of `wchar\_t`](#):
 - PBrett explained that the goal of discussing this issue is to determine if we agree with the proposed resolution.
 - Victor expressed support for it and stated that it is consistent with previous discussions.
 - Victor noted a minor markup issue in the proposed wording; the extent of the struck text should include the trailing `>` character.
 - **Poll 3: Recommend the proposed resolution to LWG3944 "Formatters converting sequences of `char` to sequences of `wchar_t`" to LWG, after fixing the typo.**
 - **Attendees: 12**
 - **No objection to unanimous consent.**
 - Mark asked what the next step is for this issue.
 - Tom advised sending the proposed resolution to the LWG chair and stated that he would work with the LWG chair to get a github issue filed to record the SG16 poll.
- Tom stated that the next meeting is scheduled for 2023-08-09.
- Zach indicated that he could have a revision of [P2728 \(Unicode in the Library, Part 1: UTF Transcoding\)](#) available by then.

- Victor reported that he has a new revision of [P2845: Formatting of std::filesystem::path](#) available.

August 23rd, 2023

Draft agenda:

- [P2909R0: Dude, where's my char?](#).
- [P2728R6: Unicode in the Library, Part 1: UTF Transcoding](#).

Attendees:

- Fraser Gordon
- Hubert Tong
- Mark de Wever
- Peter Brett
- Robin Leroy
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2909R0: Dude, where's my char?](#):
 - Much appreciation was expressed for the clever paper title.
 - *[Editor's note: in later revisions, the R0 title was demoted to a sub-title and a new title introduced; "Fix formatting of code units as integers".]*
 - Victor introduced the paper:
 - When `std::format()` was introduced, non-portable behavior due to the implementation-defined signedness of `char` was not intended.
 - It is possible that some users expect the signedness to be reflected in the output, but most users that are formatting character types as integers are intending to expose bit patterns.
 - This is technically a breaking change.
 - This is more LEWG territory, but since it is text related, it seemed prudent to collect input from SG16.
 - PBrett requested that section 2, "Proposal", be expanded to illustrate the before/after effects for each of the type options.
 - Victor agreed to do so.
 - Victor explained that the change increases compatibility with `std::printf()` for the impacted type options other than "d"; the "%d" `std::printf()` conversion specifier

always treats its argument as a signed type, but the proposed change for the "d" type option will always treat `char` as an unsigned type regardless of whether it is signed.

- Zach expressed appreciation for symmetry and that the change improves support for portable roundtripping behavior.
- Mark acknowledged that the change is a breaking change and asked if the intent is to handle this as a DR.
- Victor replied that LEWG will decide that and that he would recommend handling this as a DR.
- Mark observed the lack of a feature test macro.
- Victor stated that he could add one.
- Hubert requested that a more descriptive title be used for the paper.
- Hubert noted that it is implementation-defined whether `wchar_t` is a signed type as well.
- Victor replied that it would be reasonable to treat all `charT` types as being unsigned.
- PBrett requested that the paper be updated to explicitly mention `wchar_t` as well.
- Hubert expressed some concerns over the proposed change; `char` and `wchar_t` do have a signedness and it isn't good for programmers to ignore that.
- Victor replied that, for `wchar_t` at least, the concern is not as strong since programmers don't tend to use `wchar_t` as an integer type as is done with `char`.
- Hubert suggested it might make sense for the "d" type option to maintain signedness.
- Victor stated a preference for the signedness handling being consistent across the type options.
- Tom noted that `int8_t` could be implemented in terms of `char`.
- Hubert noted that most of the changes increase consistency with `std::printf()` and stated the improved consistency should be extended to all of the integer types.
- PBrett reminded the group that `char` is a distinct type from `signed char` and `unsigned char`.
- Zach asserted that it is surprising to get a negative value for a `char` type and stated that negative `char` values are a wart in the language.
- Hubert noted that [\[basic.fundamental\]p11](#) specifies that `char` is an integer type.
- PBrett asked if an LWG issue should be raised regarding whether `int8_t` can use `char` as its designated type.
- Fraser responded that cv-qualified types are also integer types and might therefore possibly be used as the designated type unless the `int8_t` wording excludes them.
- Hubert noted that cv-qualified types being integer types was a recent CWG change.
- PBrett reported that [\[cstdint.syn\]](#) specifies that `int8_t` must designate a signed integer type and that [\[basic.fundamental\]p1](#) doesn't include `char` in its definition of *signed integer types*.
- PBrett stated that we will file a LWG issue to clarify this.
- Tom asked for confirmation of the behavior for integer types other than `char` when used with the "o", "x", and "X" type options.

- Victor replied that negative values may be produced.
- Hubert stated that includes `wchar_t` when it is a signed type.
- Tom noted that is consistent with the status quo wording.
- Hubert noted that the wording is applicable to `charT`, but not to mixed character types.
- **Poll 1: Modify P2909R0 "Dude, where's my char?" to maintain semi-consistency with `printf` such that the 'b', 'B', 'o', 'x', and 'X' conversions convert all integer types as unsigned.**
 - Attendees: 8 (1 abstention)
 - SF F N A SA
1 2 0 2 2
 - No consensus.
 - SA: I'm not opposed to that direction in principle, but it is a deeper change and needs more research.
 - A: I'm concerned about the lack of implementation experience.
- **Poll 2: Modify P2909R0 "Dude, where's my char?" to remove the change to handling of the 'd' specifier.**
 - Attendees: 8 (1 abstention)
 - SF F N A SA
2 1 2 1 1
 - No consensus.
 - SA: That would add a corner case to a corner case; this is more LEWG territory and will get discussed there.
- **Poll 3: Forward P2909R0 "Dude, where's my char?", amended with a descriptive title, an expanded before/after table, and fixed `CharT` wording, to LEWG with the recommendation to adopt it as a Defect Report.**
 - Attendees: 8 (1 abstention)
 - SF F N A SA
2 2 2 1 0
 - Weak consensus.
- Tom asked if there are any concerns beyond the `std::printf()` inconsistencies that would motivate the N and A voters towards F/SF.
- No other concerns were raised.
- Hubert expressed unhappiness with the "d" type option direction since it won't provide help to those debugging issues related to `char` being a signed type.
- [P2728R6: Unicode in the Library, Part 1: UTF Transcoding](#):
 - Zach introduced the changes made in recent revisions:
 - The type unpacking mechanism was reworked.
 - The `null_sentinel_t` type was moved to the `std` namespace.
 - A `std::ranges::project_view` was introduced based on SG9 (Ranges) feedback though this view is likely to be replaced in a future revision with a conditionally borrowed `transform_view`.

- The `utfN_views` are now just aliases of a `utf_view` class template specialization.
- PBrett asked if anyone has new SG16 concerns inspired by the changes since R3.
- [*Editor's note: SG16's last review of this paper was [P2728R3](#) during the [2023-05-10 SG16 meeting](#).]*
- No new concerns were raised.
- Tom asked for specific ideas on how to improve presentation in the motivation section of the paper to address any lingering concerns from reviews of previous revisions.
- PBrett stated that the paper has improved significantly from previous revisions
- PBrett volunteered to meet with Zach offline to more thoroughly review that section.
- Fraser asked whether support for the `approximately_sized_range` concept proposed by [P2846 \(size_hint: Eagerly reserving memory for not-quite-sized lazy ranges\)](#) has been considered.
- Zach replied that he has to some extent and noted that there are range limits that could be imposed and that might work with that feature.
- Fraser asked if the proposal could be retrofitted to support that feature as it progresses through the committee.
- Zach replied affirmatively and explained that the `size_hint()` member could be conditionally enabled when size information is available.
- Hubert requested clarification regarding the request for improvements to the motivation section.
- Tom explained that he had received input from multiple people that they felt the motivation section was lacking.
- PBrett explained that one of the perceived issues was the lack of rationale for the design decisions made and an analysis of alternatives considered; for example, during previous SG16 discussions, vague comments were sometimes made regarding the design being motivated by performance concerns, but the performance goals and concerns are not reflected in the paper.
- PBrett repeated his earlier claim that recent revisions and the refined scope have improved the situation.
- Hubert stated that it sounds like the motivation question might be resolved then.
- Hubert suggested that a scope section could be added.
- PBrett reported that, in a recent UK body discussion concerning the failure for some papers to attain consensus, observations were made that lack of a common understanding of the problem to be solved likely contributed to the failure.
- PBrett opined that discussion of earlier revisions of the paper exhibited some confusion regarding which problems this paper is intended to address.
- Zach stated that he has been working on prototypes that lead to this paper for about seven years now and that some of the design motivation is influenced by things he learned along the way, but that would require some reflection to recall.
- Zach suggested that discussion move towards error handling as discussion of that topic was requested in the meeting agenda.

- *[Editor's note: Zach was referring to requests made on the SG16 mailing list. See <https://lists.isocpp.org/sg16/2023/08/3930.php>.]*
- Robin added some background for the linked [PR-121 \(Recommended Practice for Replacement Characters\)](#) policies. That policy paper was used to inform the recommendation made by the UTC during the [UTC 116 / L2 213 Joint Meeting held in Redmond, WA from August 11-15, 2008](#) in which a [consensus to prefer policy 2](#) was established.
- Zach reported having been unaware of PR-121 and that his design decisions were guided by what appears in the Unicode Standard.
- Zach summarized the error handling options described by the Unicode Standard as:
 - terminate
 - report an error
 - substitute a replacement character.
- *[Editor's note: the Unicode 15 chapters that discuss handling of ill-formed code unit sequences are:

 - 3.9, Unicode Encoding Forms, U+FFFD Substitution of Maximal Subparts.
 - 5.22, U+FFFD Substitution in Conversion.
]*
- Zach stated that an option to just drop ill-formed code unit sequences seems misguided.
- Robin agreed and stated that doing so can lead to security issues.
- Zach stated that there are other options to identify encoding errors and that he does not want this feature to be made complicated.
- PBrett asserted a need for a feature to just validate that a given string can be successfully decoded.
- Zach responded that such a feature was in a previous revision of the paper, but that it was removed as part of reducing scope.
- PBrett stated that he actually wants that feature more than he wants transcoding support so that input could be proactively rejected.
- Tom expressed sympathy for Zach's perspective but stated a preference towards not providing an error handler at all over providing one that is unable to handle arbitrary complexity.
- Zach replied that he really only cared to support terminate, throw, and substitute as recommended by the Unicode Standard.
- Tom described the error handling approach that JeanHeyd developed for his work on [P1629 \(Standard Text Encoding\)](#); it allows for the current iterator to be moved to an error handler that manipulates it as necessary and then moves it back; this provides the error handler full autonomy.
- Zach replied that such an approach doesn't work for a transcoding iterator since exactly one output code unit must be produced; or would otherwise require a buffer to be persisted and referenced for later outputs.
- Tom expressed gratitude for that response and reported that he had not considered the limitations of lazily transcoding within iterator operations.

- PBrett provided a brief introduction to the [ztd.text](#) error handlers.
- [*Editor's note: see the error handlers in the header files included by https://github.com/soasis/text/blob/main/include/ztd/text/error_handler.hpp.]*
- Zach noted that each iterator dereference has to produce the next code unit value and that makes it expensive to support anything other than substitution of a single code point.
- PBrett asked if more design space options are opened by considering views rather than iterators.
- Zach replied that the iterators are stateful in either case.
- Zach stated that he would be ok with dropping the error handler in favor of only doing substitution and noted that the error handler can only be specified when the iterators are used directly; the views don't support providing an error handler.
- Tom asked Hubert if his previously expressed interest in exposing the type unpacking behavior has been satisfied.
- Hubert did not recall his previous interest.
- Tom explained his recollection; that Hubert wanted to be able to take advantage of the unpacking behavior when writing adapters to be used in range pipelines.
- Zach stated that the concepts in the paper might need to be refined a bit but that he has a test that does that.
- Tom requested that an example be added to the paper.
- Hubert suggested that the motivation section be updated to explain that functionality as well.
- Tom reported that the next meeting will be 2023-09-13 and that likely agenda items include continued review of P2728R6 and initial review of [P1729R2 \(Text Parsing\)](#).

September 13th, 2023

Draft agenda:

- [P2845R3: Formatting of std::filesystem::path:](#)
 - Continue review.
- [P2728R6: Unicode in the Library, Part 1: UTF Transcoding:](#)
 - Continue review.

Attendees:

- Corentin Jabot
- Eddie Nolan
- Fraser Gordon
- Hubert Tong
- Jens Maurer

- Nathan Owen
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2845R3: Formatting of std::filesystem::path](#):
 - *[Editor's note: D2845R3 was the active paper under discussion at the telecon. The agenda and links used here reference P2845R3 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Tom noted that SG16 previously reviewed P2845R0 and that the current revision addresses prior review feedback.
 - Victor provided an introduction:
 - The recent revisions correct some minor mistakes.
 - The proposed default format now produces a non-quoted non-escaped representation.
 - If the format specifier includes the ? option, then a quoted escaped representation is produced.
 - The [{fmt} library](#) had previously implemented the behavior proposed in P2845R0 but was recently changed to implement the behavior introduced in P2845R2; this was a breaking change that impacted a few users.
 - Eddie pointed out an incorrect word choice in section 6, Proposal; "loose" is used where "lose" is intended.
 - Victor stated that the output shown for the first lone surrogate example in section 6, Proposal, might be incorrect and that he needs to check if a \x escape should be produced instead of the \u escape currently presented; the intent is for the behavior to match what is specified in [\[format.string.escaped\]](#).
 - **Poll 1: Forward P2845R2, Formatting of std::filesystem::path, to LEWG with a recommended target of C++26.**
 - **Attendees: 9 (1 abstention)**
 - **SF F N A SA**
5 2 1 0 0
 - **Strong consensus.**
- [P2728R6: Unicode in the Library, Part 1: UTF Transcoding](#):
 - Tom reminded the group that our purpose as a study group is to assist paper authors in producing a proposal that has the best possible chance of passing review in other groups and, ultimately, an adoption poll in a plenary session.
 - Tom stressed that questions asked during discussion likely reflect some lack of clarity in the paper and should therefore inspire additional edits, not just immediate

responses.

- Tom listed the topics for discussion as presented in the [previously communicated agenda](#):
 - Continued review of error handling.
 - Opportunities for simplification; whether `std::uc::format` is still needed.
 - How the proposed features fit with the bigger picture and vision for future library proposals.
- Zach summarized some other recent reviews:
 - SG9 has reviewed the paper three times and plans to review it at least one more time.
 - SG9 has provided some feedback that has not yet been incorporated in the paper.
- Zach provided an overview of the proposed error handling:
 - The error handler receives an unspecified diagnostic message that programmers can use as they please.
 - The default error handler ignores the diagnostic message and returns a replacement character.
- Zach stated that the error handling approach used in JeanHeyd's work doesn't fit into the iterator model.
- Zach said he does not have strong opinions about the error handler and suggested it could be removed for now and added back later if needed.
- Steve noted that transliteration is a use case for error handlers that allow multiple characters to be substituted for a non-translatable character.
- Steve acknowledged that support for multi-character substitution would require a buffer that would make iterators larger and more complicated.
- Corentin expressed confusion regarding the transliteration example since the proposed functionality is specific to UTF encodings.
- Corentin suggested implementing transliteration as a layered view adapter.
- Corentin stated that there are few good options beyond a single replacement character approach and that this approach is conforming, implementable, and useful.
- Victor observed that the error handling interface treats all errors the same and that there is no distinction between different kinds of errors.
- Zach responded that there are a fixed number of error possibilities and that a set of error codes would be isomorphic to the set of messages that are passed in the reference implementation.
- Tom asked Zach if he has a strong preference.
- Zach replied that his preference is to just substitute a replacement character and otherwise ignore any errors.
- Tom expressed a belief that there are other reasonable error handling possibilities, but that a concrete proposal should be offered before trying to engage further on such discussion.
- Jens noted that passing an error message creates an internationalization concern and expressed a preference against a message based interface.

- Jens pondered the plausible options for reacting to an error:
 - The substitution approach is easily understood and Unicode provides a recommendation for how to perform them.
 - Throwing an exception is easily understood and works with ranges and iterators.
 - Since the error handler is not given access to the ill-formed code unit sequence, there is little context for doing anything else useful.
- Zach indicated that he would be fine with replacing the `std::string_view` parameter with an enumeration.
- Zach noted that substitution of a different replacement character would not be inline with Unicode recommendations.
- Jens stated that there is no universal right answer for error handling for all programs; throwing an exception could be the right choice for one while aborting could be the right choice for another.
- Zach suggested that `utf_iterator` could be used to single step through the input and noted that it provides access to the underlying sequence.
- Zach stated that trying to provide a tool that handles every situation seems unnecessary.
- Jens asked if the paper has an example that demonstrates single stepping through an input sequence with access to the underlying code unit sequence.
- Tom commented that such an example would be a great addition to the paper.
- Zach asked for a poll to remove the error handler.
- Steve seconded Jens' preference to not use a stringly-typed error message.
- Steve expressed tenuous consent for removal of the error handler based on the discussion.
- Steve summarized the design requirements; there is one error handling mode that we want for sure, and a few others that we might want rarely.
- Steve expressed a desire for more examples in the paper.
- Corentin stated that views are not designed for error handling; they are designed for composition.
- Corentin outlined a design approach for a different feature; a view that iterates over ill-formed code unit sequences.
- Corentin pondered which replacement policy should be used.
- Zach replied that Unicode specifies the recommended replacement policy.
- Jens requested that the paper explicitly reference that policy.
- Zach responded that it already does.
- *[Editor's note: section 5.4, "Add the transcoding iterator template", has a paragraph that states:*

The number and position of the error handler invocations should use the "substitution of maximal subparts" approach described in Chapter 3 of the Unicode standard.

- Hubert asked if formal wording is available.
- Zach replied that there is some pseudo wording, but no real wording yet.
- Eddie observed that removal of the error handling interface would require use of a for statement for even minimal error handling.
- Eddie posited a use case that requires counting the number of errors encountered and noted that it could be implemented using the proposed design with the caveat that it would require global state since error handler objects are not persistent.
- Zach acknowledged such use cases, but characterized them as examples of corner cases that are not frequently needed.
- Hubert stated that he does not see a reason to prohibit the easy error handling cases and opined that removal of the error handler would be an over reaction.
- Tom noted that the error handler is stateless; an object of the error handler is constructed on demand for each error and since the error handler is not passed details of the error encountered, error handling is severely constrained.
- Jens characterized the lack of persistence of an error handling object as a design defect; global state should not be required for Eddie's example.
- Zach stated that the discussion has made it clear that we don't have a good grasp of the design that we want.
- Corentin pondered how error handling in the middle of a lazy algorithm should be performed and suggested that may be a question for SG9.
- Corentin stated that there are currently no standard views that throw.
- Corentin suggested that views might not be the appropriate utility to use to sanitize input.
- Corentin noted that a requirement to maintain state might make it difficult to match range complexity requirements.
- Corentin advised using a view that operates on code units to analyze code units.
- Zach agreed with Corentin's comments and stated that the proposal does not support custom error handling for views for exactly those reasons.
- Tom suggested meeting with SG9 to discuss error handling in range pipelines.
- Zach agreed to do so.
- Eddie noted that exceptions are the only way to alter the control flow in range pipelines.
- Tom agreed and stated that iterator operations don't provide an option other than throwing exceptions.
- Eddie indicated that those limitations make him less inclined to support custom error handling.
- Jens acknowledged that existing standard views might not throw exceptions directly, but noted that views that accept a callable, like `std::ranges::transform_view` allow an exception to be conditionally thrown based on the input in order to break the pipeline processing.
- Jens concurred with having SG9 weigh in and perhaps poll error handling for ranges.
- Jens requested an example of how error checking and handling could be performed using the proposed iterators in a for statement so that he could better determine how

programmers could provide their own exception throwing view.

- Zach provided an example in the chat.

```
auto v = my_view();  
for (auto it = v.begin(); it != v.end(); ++it) {  
    if (is_replacement_character(*it)) ...  
}
```

- Jens pointed out that the example doesn't differentiate between a substitution character in the input vs a substitution made due to an ill-formed code unit sequence.
- Zach replied that doing so would require inspecting and comparing code units.
- Hubert opined that we don't have a great story here if we're going to be telling programmers to figure this out themselves.
- Zach suggested that we don't have a good understanding of the needs right now, but stated improvements can be made later.
- Tom expressed skepticism that error handling could be added later since the addition of a template parameter, even one with a default argument, would break passing these class templates as template template parameters.
- Jens expressed a belief that the standard reserves the right to make such additions.
- Zach agreed that such changes would constitute an ABI break.
- Hubert agreed that it seems that we don't know exactly what we want.
- Jens expressed skepticism that there aren't compelling use cases for custom error handling and suggested that a more complete design that addresses those use cases might subsume the use cases met with what is proposed.
- Jens stated that his personal approach is to never accept bad data provided via a network since bad data can lead to security vulnerabilities.
- Zach suggested that a good solution might be to wrap the for statement that single steps the character decoding in a convenient interface for users.
- Tom advised that we not poll this topic for now and that we move on to other topics such as whether the `std::uc::format` enumeration is still needed.
- Zach stated the enumeration is no longer needed and can be removed.
- Zach backtracked slightly and stated that he needs to implement that removal to confirm that is the case.
- Jens noted that the enumeration is isomorphic to the three UTF code unit types.
- Tom directed the discussion towards another topic; how the proposed functionality fits in with other features we expect to provide in the future.
- Corentin stated that the requirements are different with respect to JeanHeyd's work on [ztd.text](#) and [P1629 \(Transcoding the !\[\]\(687b6c142f51ac6f390f8bd444e38d03_img.jpg\) - Standard Text Encoding\)](#).
- Corentin noted that Zack's proposed features are suitable for freestanding and can therefore run on a toaster; that isn't the case for JeanHeyd's work.
- Corentin expressed support for having multiple solutions, particularly since JeanHeyd's work cannot reasonably be implemented to work with the same constraints.
- Jens recalled that JeanHeyd's work is targeting WG14 and C interfaces.

- Tom replied that JeanHeyd has proposals targeting both WG14 and WG21 and that the WG14 proposal provides low level functionality needed for his WG21 proposal.
- [*Editor's note: JeanHeyd's related proposal for WG14 is [N3095 \(Restartable Functions for Efficient Character Conversion, r11\)](#).]*
- Jens observed that Zach's proposal is limited to support for the well-defined UTF encodings and avoids the complications of table lookups and such that come with support for arbitrary encodings.
- Tom asked Zach to confirm that there is no known reason that his proposal should not be implementable for freestanding implementations.
- Zach confirmed.
- Jens noted that there are no system calls involved.
- Zach reported that he spent time looking at how ztd.text is implemented and how the features could be integrated and that, the more he discussed with Tim Song, the more it looked like integration just wasn't feasible.
- Zach stated that his proposal implements lazy algorithms where as JeanHeyd's work implements eager transformations that can optimize based on knowledge of the destination.
- Zach said he could not find a reasonable way to make these compatible or to implement one in terms of the other.
- Tom claimed it is ok if these features are complementary without being integrated.
- Tom stated that the next meeting is scheduled for 2023-09-27 and that the anticipated agenda will include an initial review of [P1729R2 \(Text Parsing\)](#).
- Tom asked for opinions regarding what else should be discussed before we're ready to poll forwarding this paper.
- Corentin replied that we could work on improving the presentation in the paper for LEWG.
- Corentin noted that LEWG is likely to return the paper to SG16 for any Unicode questions not answered in the paper.
- Hubert suggested that LEWG review the design before substantial effort is expended on formal wording.

September 27th, 2023

Draft agenda:

- [P1729R2: Text Parsing](#):
 - Initial review.

Attendees:

- Eddie Nolan
- Elias Kosunen

- Fraser Gordon
- Nathan Owen
- Peter Brett
- Robin Leroy
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Tom announced that Steve Downey has agreed to take on a SG16 co-chair role and will likely participate in a SG16 meeting chair rotation.
- PBrett stated that he might have less time available for SG16 meetings in the near future due to commute changes.
- Tom reported that an in-person meeting for SG16 is not planned for Kona.
- PBrett noted that SG21 (contracts) is likely to reduce both his and Tom's availability during the Kona meeting.
- Fraser asked if SG16 is impacted by the recent decision to disallow ISO and INCITS from hosting joint meetings.
- PBrett responded that it is not.
- [P1729R2: Text Parsing](#):
 - Elias offered an introduction:
 - SG16 reviewed P1729R0 during the in-person meeting in Cologne.
 - P1729R1 was reviewed by LEWG-I soon after, but activity then stalled until recently.
 - There have been a lot of changes.
 - `std::scan` is the parsing analog to `std::format`.
 - Elias proceeded with presenting the paper.
 - PBrett pointed out an error in the comments in the example code in section 3.1, "Basic example"; `result.begin()` should be `result->begin()`.
 - Elias reported that that error has already been corrected in an R3 draft.
 - PBrett asked whether `begin()` reflects the start of the parsed range or the start of the remaining text.
 - Elias replied that it reflects the unparsed remainder.
 - Fraser asked why the text to be parsed is passed before the format string.
 - Elias replied that the order is consistent with `scanf()`.
 - Tom observed that the dereference of `result` in the example code in section 3.5, "Alternative error handling", is unconditional and would presumably result in some kind of bad behavior if the scan was not successful.
 - Elias confirmed that would result in undefined behavior, noted that an exception would be thrown if `result.value()` was used instead, and stated that the null-coalescing example avoids the bad behavior.
 - Elias stated that the example should probably be updated.

- Elias presented 3.6, "Scanning an user-defined type" and stated that, with regard to encoding, his implementation assumes UTF-8, but that doing so is probably not acceptable for the standard.
- Victor replied that most of the matching against the format string is encoding agnostic and just needs to match code units.
- Victor noted that encoding issues are relevant for cases that involve locale.
- Victor asserted that the same encoding approach should be used as for `std::format`; use the ordinary literal encoding.
- PBrett reported that other encodings, such as EUC-KR, are still widely used in some regions.
- Elias presented 6.2, "scanf-like [character set] matching", a future extension to support matching a range of characters and asked about the potential to unintentionally close off the possibility of future support for such extensions if they are not provided up front.
- Tom replied that he didn't have any such concerns.
- Tom stated that this is regex-like behavior that could be layered on in a different manner.
- PBrett expressed a desire for such a feature in order to ease migration from `scanf()`.
- PBrett noted that matching a range could be locale dependent.
- Victor asserted that feature additions should be motivated by usage and noted that `std::format()` does not provide replacements for all features in `printf()`.
- Eddie asked if a range of characters might be sensitive to encoding; for example, the meaning of the range `[A-Z]` could potentially be different for EBCDIC vs ASCII.
- Tom suggested revisiting such concerns with locale in mind at a later time.
- Tom noted that character set ranges are locale sensitive in utilities like `grep`.
- Tom directed discussion back to section 4.2, "Format strings", and stated that the use of `std::isspace()` to scan whitespace is problematic because it is only able to recognize whitespace characters that are encoded as a single code unit.
- Tom asked Victor to confirm that a definition of whitespace characters was not required for `std::format()`.
- Victor confirmed.
- Tom suggested that, if the associated literal encoding is a standard unicode encoding form, then the set of whitespace characters should be defined to match one of the Unicode whitespace definitions and that the set is otherwise implementation-defined.
- Elias noted that Unicode specifies a lot of whitespace characters and wondered how surprising recognizing them might be.
- Robin stated that Unicode provides multiple whitespace character definitions for use in various contexts via the `white_Space` and `Pattern_White_Space` character properties.
- Robin noted that `Pattern_White_Space` has the advantage of being immutable and that it includes some invisible characters that should be ignored.

- Robin explained that `Pattern_White_Space` is intended to be used for the specification of programming languages and that Unicode offers a recommendation in [UAX #31 of Unicode 15.1](#).
- [Editor's note: See [section 4.1, "Whitespace"](#) and, in particular, [conformance requirement UAX31-R3a](#).]
- Tom provided a Unicode utilities link that lists all the characters with `Pattern_White_Space=Yes`.
 - [https://util.unicode.org/UnicodeJsps/list-unicodeset.jsp?a=\[:Pattern_White_Space=Yes:\]](https://util.unicode.org/UnicodeJsps/list-unicodeset.jsp?a=[:Pattern_White_Space=Yes:])
- PBrett lamented the "null" names for the C0 and C1 control characters.
- Robin filed an issue to correct the display names.
 - [The UnicodeSet utility should show Name_Alias in the absence of Name](#)
- [Editor's note: Following the meeting, Robin shared additional details regarding the categorization of whitespace in the Unicode Standard. Even this is an incomplete list.
 - [Things that are categorized as space separators](#)
 - [Things that are called spaces](#)
 - [Things that are white space](#)
 - [Things that should be treated as white space in machine-readable syntaxes](#)
 - [Things that behave like space for line breaking](#)
 - [Things that behave like space for bidirectional ordering](#)
 - [Things that behave like space for word separation](#)
]
- Tom asked for a clarification in the example in section 4.3.2, "Fill and align"; if the format string for `rk` was changed to `"**42"` would that result in an error like in the `rI` example?
- Elias replied affirmatively.
- Tom suggested it might be worth adding an additional example to make that explicit.
- Elias asked what the behavior should be for input that is invalidly encoded and stated that he had planned to propose substitution with a replacement character, but that approach is problematic for output types with reference semantics like `std::string_view`.
- Tom asked facetiously why the input wasn't sanitized before being passed to `std::scan`.
- Tom noted that this is a basic error handling question.
- Victor stated that substituting a replacement character doesn't work well in general since it can't be matched by most of the type specific scanners.
- Victor suggested treating invalidly encoded input as an error such that an unsuccessful scan result is returned.
- PBrett noted that ill-formed sequences could just be passed through when scanning for string input.
- Tom asked how the scanner would know when to stop scanning.

- PBrett replied that the replacement character could be handled as a character that doesn't match any other character.
- Tom quipped, so it's a NaN.
- PBrett agreed.
- Robin suggested treating such substituted characters as the unknown character, U+FFFF, with regard to Unicode properties and shared a link that lists those properties.
 - <https://util.unicode.org/UnicodeJsps/character.jsp?a=FFFF&B1=Show>
- Elias expressed concern about the overhead imposed by always validating the encoding of the input, but noted that passing ill-formed sequences through means that errors are sometimes caught and sometimes not.
- Eddie expressed a desire for input to be sanitized to avoid having to worry about the consequences otherwise.
- PBrett observed that, historically, we have required input to be sanitized and noted that validating the input is helpful to avoid undefined behavior.
- PBrett summarized the options available; make the behavior well-defined, make it undefined, or, perhaps, categorize it as erroneous though we don't yet know how to apply that concept to the standard library.
- *[Editor's note: "Erroneous behavior" is a concept recently discussed within WG21 in the context of [P2795 \(Erroneous behaviour for uninitialized reads\)](#).]*
- Elias noted that scanners written for user-defined types are unlikely to perform encoding validation, but that we could encourage authors to delegate scanning back to `std::scan` as demonstrated in section 3.6, "Scanning an user-defined type".
- Robin reported that Unicode does have a conformance requirement that ill-formed input not be treated as a character and directed the group to [conformance clause C.10 in chapter 3 of Unicode 15](#).

When a process interprets a code unit sequence which purports to be in a Unicode character encoding form, it shall treat ill-formed code unit sequences as an error condition and shall not interpret such sequences as characters.

- PBrett interpreted the clause as motivation for erroneous behavior.
- Robin asked whether erroneous behavior is similar to Ada's concept of bounded errors and provided a link to [section 1.1.5, "Classification of Errors" in the Ada 2022 standard](#).
- *[Editor's note: "erroneous behavior" as recently used in WG21 does appear to correlate well with Ada's "bounded errors". Note that Ada's "erroneous execution" corresponds to the C and C++ notion of "undefined behavior".]*
- Tom provided a brief overview of the recent history of erroneous behavior and its proposed use for reads of uninitialized variables.
- *[Editor's note: Robin shared a link to [section 13.9.1, "Data Validity", paragraph 9 in the Ada 2022 standard](#) and its discussion of handling of objects with invalid representations; such cases might arise due to lack of initialization.]*
- PBrett asked if there is a way to scan a single code point.

- Elias replied that there is in his reference implementation but that it is provided via a distinct scanner specialization for a `code_point` type.
- Elias suggested that it might be desirable to support transcoding of `charN_t`-based types in the future.
- PBrett noted that scanning of `charN_t`-based types does not involve ambiguous encoding.
- Victor expressed uncertainty regarding how to handle such transcoding when the corresponding literal encoding isn't a standard Unicode encoding form.
- Elias stated that a programmer can handle such concerns on their own, but only for a user-defined type since they would not be permitted to specialize `std::scanner` for the `charN_t` types.
- Elias explained that locale support is opt-in the same as it is for `std::format` and that the classic locale is used by default.
- Tom pondered whether it would be desirable to recognize input using both the specified locale and the classic locale.
- PBrett expressed strong opposition.
- Tom realized use of multiple locales doesn't work at all because recognition of characters used for, e.g., thousands separators and decimal points would be ambiguous.
- PBrett requested the addition of an example to the paper to demonstrate explicit use of a locale.
- PBrett asserted that it is a programmer requirement to ensure that input is correctly encoded for the specified locale.
- Elias directed attention to section 4.3.5, "Localized (L)", and asked whether a misplaced grouping separator should result in an error.
- Elias indicated that the proposed behavior is consistent with `iostreams`.
- Victor cautioned against being innovative and opined that existing practice should be followed.
- Victor noted that implementation of a relaxed scanner should not be difficult when needed.
- Tom noted the discussion of alternative options for interpretation of field width units in section 4.3.4, "Width and precision" and asked for motivating reasons to consider options that differ from `std::format`.
- Elias replied that the proposed behavior differs from `std::scanf()`.
- Tom asked whether the message strings described in section 4.6, "Error handling" require translation or localization.
- Elias replied that they are intended for use with `std::exception` and therefore target programmers rather than end users.
- Tom stated that the next meeting is scheduled for October 11th and that an agenda is still to be determined.